Order Documentation Tracker Code Intuition Report

How the app works, following the code

Version	2.0 (Vercel edition)
Covers	lib/app.js, public/index.html, api/index.js, server.js, reset-admin.js
Audience	Developers and maintainers new to the project
Prepared	October 2026

Contents

1. How to read this report **4**

1.1 The mental model in one paragraph . . . 4

1.2 The two code files that matter . . . 4

1.3 Notation . . . 4

2. The big picture: two journeys through the code **4**

2.1 Journey A - opening the site . . . 4

2.2 Journey B - a change (example: passing a milestone) . . . 5

3. Backend walkthrough (lib/app.js) **5**

3.1 Entry points . . . 5

3.2 Imports, limits and the database pool . . . 5

3.3 init() - preparing a fresh database . . . 5

3.4 Settings: kvGet, kvSet, DEF, getCfg . . . 6

3.5 Passwords . . . 6

3.6 Sessions: sign, unsign, cookies, setCookie, userFrom, issue . . . 6

3.7 Client IP and login throttling . . . 7

3.8 Validation helpers . . . 7

3.9 diffLog(o, n, who) - the audit trail . . . 7

3.10 Reading and writing orders safely . . . 7

3.11 HTTP helpers . . . 8

3.12 api() - the router, endpoint by endpoint . . . 8

3.13 The exported handler . . . 9

3.14 reset-admin.js . . . 10

4. Frontend walkthrough (public/index.html) **10**

4.1 Page shell . . . 10

4.2 Boot code . . . 10

4.3 useApp() - loading and refreshing data . . . 10

4.4 ost(), cls() and Track - the status engine . . . 10

4.5 Login and Account . . . 11

4.6 The field-label system . . . 11

4.7 Multiple customer e-mails . . . 11

4.8 MsEdit . . . 11

4.9 OrderForm - create and edit . . . 11

4.10 Detail - one order . . . 12

4.11 Users component . . . 12

4.12 Settings component . . . 12

4.13 Dashboard - every calculation . . . 13

4.14 App - the shell . . . 13

5. Feature walkthroughs (end to end) **14**

5.1 Sequence: attaching a file 14

5.2 Sequence: concurrent edit conflict 14

6. Things to know when changing the code **15**

Glossary 15

1. How to read this report

This report explains **how the application works by following the code**. It is written for someone who can read basic JavaScript but has not seen this project before. Where the Technical Report lists *what* exists (folders, tables, endpoints), this report explains *why each piece of code is there and what happens when it runs*.

1.1 The mental model in one paragraph

The browser downloads one HTML file that contains a React app. The app asks the server for **everything** (`GET /api/state`: settings, the signed-in user and all orders), computes statuses itself from dates, and draws the dashboard. When the user changes something, the app sends one small request (`PUT /api/orders/:id`, `POST /api/login`, ...) and then asks for everything again. The server is a single JavaScript function that checks who you are, checks what you may do, validates the data, and reads or writes Postgres (data) and Vercel Blob (files).

1.2 The two code files that matter

File	Lines	What lives in it
`lib/app.js`	317	The whole backend: database pool, schema creation, login/sessions, validation, audit log, and every API endpoint.
`public/index.html`	356	The whole frontend: CSS, then React components: App, Dashboard, Detail, OrderForm, Settings, Users, Login, Account and small helpers.
`api/index.js`	3	Vercel entry point; re-exports the backend handler.
`server.js`	16	Local-only web server for development.
`reset-admin.js`	17	Command-line password recovery tool.

1.3 Notation

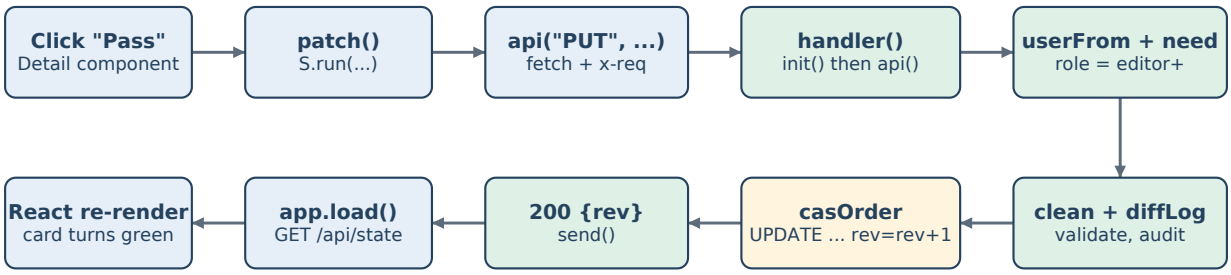
Short code fragments are quoted from the real files; "(abridged)" means lines were shortened for readability. Function names in `monospace` can be searched in the source. "Order" means one customer PO being tracked; "milestone" means one document deadline inside an order.

2. The big picture: two journeys through the code

2.1 Journey A - opening the site

- **Browser -> `/`.** Vercel's CDN returns `public/index.html`. The `<script>` tags download React, ReactDOM and Babel from cdnjs.
- **Babel compiles** the `<script type="text/babel">` block (the JSX) in the browser, then the last line `ReactDOM.createRoot(...).render(<App/>)` starts the app.
- `App` **calls** `useApp()`, whose `useEffect` runs `load()` immediately. `load()` calls `api("GET", "/api/state")`.
- **Vercel routes** `/api/state` to the function (rewrite in `vercel.json`). `api/index.js` passes the request to `lib/app.js`.
- `handler()` awaits `init()` (creates tables on a cold start), then runs `api()`. `api()` reads the cookie (`userFrom`), loads settings (`getCfg`), and reaches the `GET /api/state` branch.
- **If the dashboard is private and nobody is signed in,** the server answers 401 with the app name and colour; the UI shows the sign-in form. **Otherwise** it returns `{me, cfg, orders}`.
- `useApp` **stores the result** in React state; `Dashboard` computes statuses with `ost()` and draws cards, bars and order rows. A timer repeats `load()` every 20 seconds while the tab is visible.

2.2 Journey B - a change (example: passing a milestone)



Left-to-right on the top row, then right-to-left on the bottom row. Every write ends with a full reload of state.

- The ✓ **Pass** button calls `patch(milestone, {done: TODAY})`, which wraps a `PUT /api/orders/:id` in `S.run()`.
- The request body is `{rev, data}`: the order as the browser last saw it, with that one milestone changed. `rev` is the version number the browser last read.
- On the server, `need("editor")` rejects viewers; `clean()` validates; `diffLog()` appends "W2 ...: Passed" to the history; `casOrder()` writes only if `rev` still matches.
- The server replies `{rev: n+1}`. `S.run()` then calls `app.load()`, so the browser fetches the fresh state and React redraws: the milestone block turns green and the dashboard counts change.

3. Backend walkthrough (lib/app.js)

3.1 Entry points

`api/index.js` is three lines: `module.exports = require("../lib/app")`. Vercel treats any file in `api/` as a serverless function; this one exports the request handler. `vercel.json` rewrites every `/api/*` URL to it, so one function serves all endpoints and the router inside `lib/app.js` decides what to do.

`server.js` (local development only) creates a Node `http` server: URLs starting with `/api/` go to the same handler; anything else is looked up inside `public/` (with a path-traversal guard: the resolved path must start with the `public` folder). This lets you run the exact same code on your laptop.

3.2 Imports, limits and the database pool

```
const MAXUP = (+process.env.MAX_UPLOAD_MB || 100) * 1048576;   // upload limit in bytes
const SESSION_DAYS = +process.env.SESSION_DAYS || 7;
const pool = new Pool({ connectionString: DB_URL, max: 3, idleTimeoutMillis: 10000 });
const q  = async (text, params) => (await pool.query(text, params)).rows;     // all rows
const q1 = async (text, params) => (await q(text, params))[0];                // first row
const E = (status, msg) => Object.assign(new Error(msg), { status });        // throwable HTTP error
const now = () => new Date().toISOString().slice(0, 16).replace('T', ' ');     // "2026-10-13 08:07" (UTC)
const uid = () => crypto.randomBytes(6).toString('hex');                      // 12-char random id
```

- The **pool** reuses connections between requests handled by the same function instance. `max: 3` stays within database connection limits when many instances run.
- `q`/`q1` hide boilerplate. Values are always passed in the `params` array and referenced as `$1`, `$2`; that is what prevents SQL injection.
- `E(404, "Order not found")` creates an Error with a `status` property. Anywhere in the code `throw E(...)` jumps straight to the catch in the exported handler, which converts it to an HTTP response.
- `now()` creates the timestamp text used in the audit log and the files table.

3.3 init() - preparing a fresh database

`init()` runs once per cold-started function instance. It is memoised in the variable `ready`: the first call stores the promise, later calls return the same promise, and a failure clears it (`ready = null`) so the next request retries.

```
function init() {
  return ready || (ready = (async () => {
    await q(`CREATE TABLE IF NOT EXISTS users(...)`);            // + unique index on LOWER(username)
    await q(`CREATE TABLE IF NOT EXISTS kv(...)`);  ... orders, files, login_fails ...
    SECRET = process.env.SECRET;
    if (!SECRET) {                                               // no env var: keep a generated key in the DB
      await q(`INSERT INTO kv(k,v) VALUES('secret',$1) ON CONFLICT DO NOTHING`, [JSON.stringify(randomHex32)]);
      SECRET = JSON.parse((await q1(`SELECT v FROM kv WHERE k='secret'`)).v);
    }
    if (!(await q1('SELECT 1 FROM users'))) {                    // empty users table = first run
      const pw = process.env.ADMIN_PASSWORD || randomPassword, un = process.env.ADMIN_USER || 'admin';
      const r = await pool.query(`INSERT INTO users(...) VALUES($1,'Administrator','admin',$2) ON CONFLICT DO NOTHING`, [
      if (r.rowCount) console.log('First run: admin account created ...');
    }
  })().catch(e => { ready = null; throw e; }));
}  // (abridged)
```

- `IF NOT EXISTS` makes it safe to run on every cold start; existing data is never touched.
- `ON CONFLICT DO NOTHING` protects against two instances starting at the same time: the loser simply inserts nothing. Only the instance whose insert actually happened (`rowCount` is 1) logs the generated password.
- The signing `SECRET` should normally come from an environment variable; the database fallback keeps sessions valid across instances when you forgot to set it.

3.4 Settings: kvGet, kvSet, DEF, getCfg

Application settings are a single JSON document under key `cfg` in the `kv` table. `DEF` holds defaults (app name, manufacturer, 12 weeks, colour `#24415e`, `public: true`, six default milestones at weeks 2, 4, 6, 8, 10, 12 with a 7-day reminder). `getCfg()` returns `{...DEF, ...stored}`, so a missing key falls back to its default. `api()` calls `getCfg()` on every request because it needs `cfg.public` for access control.

3.5 Passwords

```
const hashPw = pw => { const s = randomBytes(16).toString('hex'); return s + ':' + scryptSync(pw, s, 32).toString('hex');
const checkPw = (pw, stored) => {
  const [s, h] = String(stored).split(':'); if (!s || !h) return false;
  const a = crypto.scryptSync(pw, s, 32), b = Buffer.from(h, 'hex');
  return a.length === b.length && crypto.timingSafeEqual(a, b);
};
```

`hashPw` makes a random 16-byte salt and stores `salt:hash`. scrypt is deliberately slow and memory-hard, which makes guessing passwords expensive. `checkPw` recomputes the hash with the stored salt and compares in constant time (`timingSafeEqual`), so response timing reveals nothing about how many characters matched.

3.6 Sessions: sign, unsign, cookies, setCookie, userFrom, issue

```
sign({u:1, tv:0, e:expiryMs})  ->  base64url(JSON) + "." + HMAC_SHA256(base64url(JSON), SECRET)

const userFrom = async req => {
  const p = unsign(cookies(req).vt); if (!p || p.e < Date.now()) return null;   // bad signature or expired
  const u = await q1('SELECT * FROM users WHERE id=$1', [p.u]);
  return u && u.active && u.tv === p.tv ? u : null;                             // still active, same token version
};
```

- The cookie is **not encrypted, only signed**: anyone can read `{u, tv, e}` but nobody can change it without the secret. It holds no password.
- `cookies(req)` splits the raw `Cookie` header into an object. `setCookie()` writes `vt=...; HttpOnly; SameSite=Lax; Path=/; Max-Age=...` and adds `Secure` when the request came over HTTPS.
- `tv` **(token version)** is the revocation switch. Changing a password, resetting it as admin, or deactivating a user increments `tv` in the database, so every older cookie fails the `u.tv === p.tv` check on its next request.
- `pub(u)` strips a user row down to the public fields (never the password hash). `issue()` creates and sets a fresh cookie for a user.

3.7 Client IP and login throttling

`ipOf(req)` returns the first address in the `x-forwarded-for` header when running behind Vercel (`trustProxy()` is true when `VERCEL=1` or `TRUST_PROXY=1`), otherwise the socket address. The login endpoint stores failures per IP in `login_fails` and refuses further attempts after 8 failures within 15 minutes (details in 3.12).

3.8 Validation helpers

Helper	What it does	Example
`S(v, n)`	Converts to string, trims whitespace, cuts to n characters. Null becomes an empty string.	`S(" ACME ", 200)` -> `"ACME"`
`D(v)`	Accepts only `YYYY-MM-DD`, otherwise returns an empty string.	`D("10/09/2026")` -> `""`
`R(v)`	Reminder days: integer clamped to 0-365; default 7 when not a number.	`R("abc")` -> `7`, `R(999)` -> `365`
`emails(v)`	Splits by comma/semicolon/space, validates each with `EMAIL_RE`, removes duplicates (case-insensitive), max 20, returns `"a@x.com, b@y.com"`. Throws 400 on an invalid address.	`"a@x.com b@y.com;a@x.com"` -> `"a@x.com, b@y.com"`
`clean(d, old)`	Builds a safe order object from untrusted input (below).	-

clean(d, old) step by step

- Rejects the order (400) unless `date` (valid date), `orderNo` and `customer` are present.
- `extra`: at most 40 custom fields, key up to 60 characters, value up to 500.
- `labels`: keeps only the seven known keys (`date, orderNo, customer, email, est, mref, weeks`) with non-empty names up to 60 characters.
- `hide`: keeps only fields that may be hidden (`email, est, mref, weeks`). A hidden field is stored as empty (`email`, `est`, `mref` are blanked).
- `ms`: up to 60 milestones; week 0-520, title 200, reminder days via `R`, `done` a valid date or null, `note` up to 2000.
- `email`: validated by `emails()` **unless** the value is unchanged from the stored order (`old.email`). That exception lets old orders with a legacy address still be saved.
- `weeks` clamped to 1-520. Finally `files` and `log` are copied from `old` (or empty for a new order) - **never from the client**.

3.9 diffLog(o, n, who) - the audit trail

Compares the stored order `o` with the cleaned new order `n` and returns log entries `{who, at, what}`. If any header field (date, order number, customer, e-mail, delivery, reference, weeks, extras, labels, hidden fields) differs, it writes "Order details edited". Then each milestone is matched by the key `week|title`: new key -> "milestone added"; changed `done` -> "Passed" or "Pass undone"; changed note -> "remarks updated"; changed reminder days -> "reminder set to N days before due"; keys left over -> "milestone removed". **Consequence:** editing a milestone's title or week appears as one removal plus one addition.

3.10 Reading and writing orders safely

```
const getOrder = async id => { const r = await q1('SELECT * FROM orders WHERE id=$1', [id]); return r && { ...JSON.parse(
const strip = o => { const { id, rev, ...d } = o; return d; };         // order object without id/rev

const casOrder = async (id, d, rev) =>
  (await pool.query('UPDATE orders SET data=$2, rev=rev+1 WHERE id=$1 AND rev=$3', [id, JSON.stringify(d), rev])).rowCoun

async function mutate(id, fn) {                                          // read-modify-write with retries
  for (let i = 0; i < 6; i++) {
    const o = await getOrder(id); if (!o) throw E(404, 'Order not found');
    const d = strip(o); fn(d, o);
    if (await casOrder(id, d, o.rev)) return;
  }
  throw E(409, 'Order is being changed by someone else, please retry');
}  // (abridged)
```

Optimistic locking in one sentence: the UPDATE only matches when `rev` is still the value the caller read; if another user saved first, `rev` has moved on, zero rows match, `casOrder` returns false and the caller reports a conflict instead of silently overwriting. `mutate()` is used where the user did not edit the form (attaching or removing a file): it simply re-reads the newest version and applies the change again, up to 6 times.

3.11 HTTP helpers

Helper	Behaviour
`send(res, code, obj)`	Sets status, `Content-Type: application/json`, `Cache-Control: no-store`, `nosniff`, then writes the JSON string.
`readRaw(req, max)`	Collects the request stream into a Buffer; rejects with 413 if it exceeds `max`. Used by the local server.
`readJson(req)`	Returns the parsed JSON body. On Vercel the platform has often already parsed it into `req.body`, so that is used first (Buffer, string or object); otherwise it reads the stream. Bad JSON becomes 400.
`lastAdmin(id)`	True if no *other* active admin exists. Guards user deletion/demotion/deactivation.
`safeName(n)`	File name up to 200 characters with slashes, backslashes, quotes and line breaks replaced by _.

3.12 api() - the router, endpoint by endpoint

Preamble (runs for every request)

```
const m = req.method, u = await userFrom(req), cfg = await getCfg();
if (m !== 'GET' && req.headers['x-req'] !== '1') throw E(403, 'Bad request');   // CSRF guard
const need = role => {                                                          // authorisation guard
  if (!u) throw E(401, 'Login required');
  if (role === 'admin' && u.role !== 'admin') throw E(403, 'Admin access required');
  if (role === 'editor' && u.role === 'viewer') throw E(403, 'Editor access required');
};
const canRead = () => { if (!u && !cfg.public) throw E(401, 'Login required'); };
```

`u` is the signed-in user or null. `need()` is called at the top of each protected endpoint; `canRead()` guards the two read endpoints that can be public. The custom header requirement blocks forged cross-site form posts because browsers do not let another site add custom headers.

GET /api/state

If nobody is signed in and the dashboard is private, answers 401 with `{error:"login", brand:{app, mfr, ac}}` (so the login screen can be branded). Otherwise loads every row of `orders`, parses each JSON `data`, adds `id` and `rev`, and returns `{me, cfg, orders}`.

POST /api/login

- Look up the failure record for this IP. If `n >= 8` and the last failure was less than 15 minutes ago, throw 429.
- Find the user case-insensitively. If the user is missing, inactive, or the password check fails: upsert the failure counter (`n` = old count within the window + 1, `t` = now) and throw 401 "Invalid username or password" - the same message for every cause, so attackers cannot discover which usernames exist.
- On success: delete the failure record, `issue()` the cookie, return `{me}`.

POST /api/logout and POST /api/password

Logout writes the cookie with an empty value and `Max-Age=0`, which makes the browser delete it. Password change requires login, checks the old password, enforces 8+ characters, stores the new hash and increments `tv`, then issues a new cookie carrying the new `tv`: the current browser stays signed in while every other session is revoked.

POST /api/orders (create)

`need("editor")`, `clean(b.data)`, a new `uid()`, first log entry "Order created", `INSERT` with `rev = 1`. Returns `{id}` so the UI can open the new order.

PUT /api/orders/:id (update)

```
need('editor'); const old = await getOrder(g[1]); if (!old) throw E(404, 'Order not found');
const b = await body(); if (b.rev !== old.rev) throw E(409, 'Order was changed by someone else');   // quick check
const n = clean(b.data || {}, strip(old));
n.log = [...old.log, ...diffLog(old, n, u.name)].slice(-500);                                       // keep last 500
if (!(await casOrder(old.id, n, old.rev))) throw E(409, 'Order was changed by someone else');      // atomic check
return send(res, 200, { rev: old.rev + 1 });
```

There are two conflict checks: the first is a cheap early exit; the second (inside SQL) is the real protection against two requests racing each other.

DELETE /api/orders/:id

Admin only. Collects the Blob paths of the order's files, deletes those blobs in one call (errors are logged but do not stop the operation), then deletes the `files` rows and the order row.

POST /api/blob (upload token) and POST /api/orders/:id/files (register)

```
// token: browser asks permission to upload
const out = await handleUpload({ body: b, request: req,
  onBeforeGenerateToken: async (pathname, clientPayload) => {
    oid = String(JSON.parse(clientPayload || '{}').oid || '');
    if (!/^\w+$/.test(oid) || !pathname.startsWith('orders/' + oid + '/')) throw E(400, 'Bad upload path');
    if (!(await getOrder(oid))) throw E(404, 'Order not found');
    return { maximumSizeInBytes: MAXUP, addRandomSuffix: true };
  } });
```

The token endpoint is the **gatekeeper for uploads**: only editors get a token, and the token only allows writing under `orders/<id>/` for an order that exists, up to `MAXUP` bytes, with a random suffix on the file name to avoid collisions. The file bytes never pass through the function. After the upload the browser calls the register endpoint, which re-checks the path, uses `blob.head()` to confirm the blob exists and learn its true size and content type, rejects an already-registered path (409), inserts the `files` row and appends the file and a log line "Attached name" to the order through `mutate()`.

GET /api/files/:id (download)

`canRead()`, find the `files` row, `blob.get(path, {access:"private"})` to open the private blob as a stream. PDFs and common images are sent `inline`; everything else is sent as `application/octet-stream` with `attachment`, so uploaded HTML or scripts can never run in your site's origin. The loop reads chunks from the Blob stream and writes them to the response, waiting for `drain` when the client is slow (back-pressure).

DELETE /api/files/:id

Editor only. Removes the file entry from the order JSON and logs "Removed attachment name" (`mutate`), deletes the blob (best effort) and the `files` row.

PUT /api/cfg

Admin only. Rebuilds a clean settings object (names trimmed and length-limited, `weeks` at least 1, `public` as boolean, colour only if it matches `#rrggbb`, extra field names max 40, milestones max 60 with clamped numbers) and saves it with `kvSet("cfg", ...)`.

Users: GET/POST /api/users, PUT/DELETE /api/users/:id

- **List:** all users ordered by name, passed through `pub()`.
- **Create:** username must match `^[\w.@-]{3,40}$`, role must be admin/editor/viewer, password 8+ characters; `INSERT ... ON CONFLICT DO NOTHING` and a 409 if nothing was inserted (duplicate name).
- **Update:** accepts any of `role`, `active`, `name`, `password`. If the target is an admin and the change would remove the last active admin, 400. A password change or deactivation bumps `tv`, ending that user's sessions.
- **Delete:** same last-admin guard, then `DELETE`.

3.13 The exported handler

```
module.exports = async function handler(req, res) {
  try { await init(); const url = new URL(req.url, 'http://x'); return await api(req, res, url, url.pathname); }
  catch (e) {
    if (res.headersSent) { console.error(e); return res.end(); }
    if (e.status) return send(res, e.status, { error: e.message });     // expected errors from E()
    console.error(e); send(res, 500, { error: 'Server error' });         // unexpected: log, hide details
  }
};
```

3.14 reset-admin.js

Run on an operator's machine with the database URL in the environment. It hashes the given password (or a random one), updates the user's password, makes the account an active admin and bumps `tv` (ending existing sessions). If the user does not exist it inserts it. It prints the credentials once.

4. Frontend walkthrough (public/index.html)

4.1 Page shell

The top of the file has `<meta viewport>` (with `viewport-fit=cover` for phone notches), three `<script src>` tags (React, ReactDOM, Babel) and a `<style>` block. CSS variables (`--bg, --card, --tx, --ac, --gr, --rd...`) define colours; `@media (prefers-color-scheme: dark)` redefines them for dark mode. Classes such as `.card`, `.row`, `.pill`, `.track`, `.ms`, `.modal`, `.bar` are reused by all components. `@media print` hides headers and buttons for printing. The `<div id="root">` is where React mounts.

4.2 Boot code

```
const {useState,useEffect,useMemo,useCallback}=React;
const TODAY = new Date(Date.now() - new Date().getTimezoneOffset()*6e4).toISOString().slice(0,10);   // local date "YYYY-
const addD = (s,n) => { const d=new Date(s+'T00:00:00Z'); d.setUTCDate(d.getUTCDate()+n); return d.toISOString().slice(0,

async function api(m,u,b){
 const r = await fetch(u,{method:m,credentials:'same-origin',headers:{'x-req':'1',...(b?{'content-type':'application/json
 const j = await r.json().catch(()=>({}));
 if(!r.ok) throw Object.assign(new Error(j.error||r.statusText),{status:r.status,body:j}); return j }
```

- `TODAY` is the user's *local* calendar date (the timezone offset is subtracted before taking the ISO date). All due-date comparisons are plain string comparisons of `YYYY-MM-DD` dates, which sort correctly.
- `addD(date, days)` adds days to a date string using UTC arithmetic to avoid daylight-saving surprises.
- `api()` is the only function that talks to the backend (except file upload). It always sends cookies and the `x-req` header, parses JSON, and turns any non-2xx answer into a thrown `Error` that carries `status` and the parsed `body`. Every caller can therefore use `try/catch`.

4.3 useApp() - loading and refreshing data

A custom hook holding `{ready, locked, me, cfg, orders, brand}`. `load()` fetches `/api/state`; success stores everything and sets `locked:false`; a 401 sets `locked:true` and keeps `brand` (name/colour for the login page); other errors are logged. A `useEffect` runs `load()` once and then every 20 seconds, skipping ticks while the browser tab is hidden (`document.hidden`), and clears the timer when the component unmounts. This polling is how one user sees another user's changes without a manual refresh.

4.4 ost(), cls() and Track - the status engine

```
const ost = o => {
 const ms = o.ms.slice().sort((a,b)=>a.w-b.w).map(m => {
   const due = addD(o.date, m.w*7);                                  // PO date + week*7 days
   return {...m, due, s: m.done ? 'done' : due < TODAY ? 'late' : 'open'};
 });
 const cur = ms.findIndex(m => m.s !== 'done');                      // first unfinished milestone (-1 if none)
 return { ms, cur,
   soon: ms.some(m => m.s==='open' && m.due <= addD(TODAY, m.r ?? 7)), // an open milestone within its reminder window
   red:  ms.some(m => m.s==='late'),
   done: cur < 0 && ms.length > 0 };
};
const cls = o => { const s = ost(o); return s.done || !s.red ? 'green' : 'red'; };
```

Rules. A milestone is **done** if it has a pass date, **late** if not done and its due date is before today, otherwise **open**. An order is **red** if any milestone is late, **soon** if any open milestone falls within its reminder window, and **done** when every milestone is passed. `cls()` turns that into the colour class used on cards.

> **Worked example (today = 2026-10-13).** Order PO date 2026-09-10 with milestones W2 (passed 2026-09-20), W4 and W6, reminder 7 days. Due dates: W2 = 09-10 + 14 days = 09-24; W4 = +28 days = 10-08; W6 = +42 days = 10-22. W2 is **done**. W4 is not done and 10-08 < 10-13, so it is **late**. W6 is **open**; its reminder window starts when `due <= today + 7`, i.e. 10-22 <= 10-20, which is false, so it is not "due soon" yet. Result: `cur` = index of W4, `red` = true, so the order shows red with "Next: W4 ..., due 2026-10-08".

`Track` draws one small coloured bar per milestone (`<i className="done|late|open">`) with a tooltip such as "W4 Material certificates - OVERDUE, due 2026-10-08".

4.5 Login and Account

- `Login` keeps username, password, error and busy flag in local state. `go()` posts credentials, then `S.load()` refreshes the global state (which now contains `me`) and closes the modal. Enter in the password field triggers `go`. When the dashboard is private it is rendered as a full page (`close` is undefined); otherwise as a modal opened by the header Login button.
- `Account` (opened by clicking your name) calls `POST /api/password` with the old and new passwords and shows the result message inline.

4.6 The field-label system

`BASE` maps built-in field keys to default labels ("PO receipt date", "Order no", ...). `lbl(order, key)` returns the order's custom label if present (`order.labels[key]`) else the default. `hidden(order, key)` tells whether the order removed that field (`order.hide`). `REQ` lists fields that cannot be removed. `LabelEdit` is an input showing a label with `defaultValue`; on blur or Enter it commits a changed non-empty value through `onCommit`, otherwise restores the old text. Because labels live inside each order, renaming "Customer name" to "Buyer" affects only that order, in the form, the detail page and the dashboard rows.

4.7 Multiple customer e-mails

`splitEm(text)` splits by comma, semicolon or whitespace. `EmailsInput` renders existing addresses as removable chips and an input box. Typing a separator (comma, semicolon, space) or pressing Enter calls `commit()`, which splits the text, validates each with `EM_RE`, ignores duplicates (case-insensitive), caps the list at 20 and reports invalid items under the box while keeping them in the input to be corrected. Backspace on an empty input removes the last chip. The value is passed up as `"a@x.com, b@y.com"`, the same format the server stores.

4.8 MsEdit

One milestone row with: week number, title, "Remind N days before due" and a remove button. It is a controlled component: every change calls `on(field, value)` supplied by the parent. It is shared by the order form and the settings page.

4.9 OrderForm - create and edit

- **Initial state:** when editing, a copy of the order; when creating, defaults: today's date, weeks from settings, one empty extra field per name in `cfg.fields`, and milestones copied from `cfg.ms` with `done: null` and empty notes.
- **Updaters:** `set` (top-level field), `ex` (extra field value), `msu` (one milestone property), `setLbl` (rename a built-in label), `del`/`restore` (hide or restore an optional built-in field; hiding e-mail, delivery or reference also clears the value), `renameX`/`delX` (rename or delete a custom field; renaming to an existing name shows an alert).
- `fld(key, input)` wraps a built-in input with its editable label and a × button (or a required star). It is a plain function, not a component, so inputs do not lose focus on each keystroke.

- **Add field** box appends a new custom field (duplicates refused). **Removed fields** buttons restore hidden fields. **+ Add milestone** appends a row defaulting to the order's documentation week.
- `save()`: checks the three required fields, converts week and reminder values to numbers, then calls `PUT /api/orders/:id` with `{data, rev}` when editing or `POST /api/orders` when creating, all wrapped in `S.run()`. `onDone` closes the editor (edit) or returns to the dashboard and opens the new order (create).

4.10 Detail - one order

- **Header card:** title, status pill (Completed / Overdue / On track), a one-line summary of dates and fields built from the labels and extras, the `Track` strip, and buttons: Back, Edit (editors), Email reminder (editors), Print, Delete (admin, with confirm), Close.
- `patch(milestone, changes)`**:** sends the whole order with one milestone modified (matched by week and title) as `PUT` with the current `rev`. Used for passing, undoing and saving remarks.
- **Pass / Undo pass:** sets `done` to `TODAY` or back to null. A milestone passed after its due date is labelled "Passed late".
- **Remarks:** an uncontrolled `<textarea defaultValue=...>`; `onBlur` saves only when the text actually changed. Viewers see the note as plain text.
- `rem()` **- e-mail reminder:** lists every unfinished milestone with its due date (marking OVERDUE), builds a `mailto:` URL with all customer addresses, a subject and a polite body, and navigates to it so the user's mail program opens.
- `upload()`**:** (1) dynamic `import()` of the Vercel Blob client from esm.sh through `new Function("u","return import(u)")` so Babel does not rewrite it; (2) `upload("orders/<id>/<name>", file, {access:"private", handleUploadUrl:"/api/blob", headers:{"x-req":"1"}, clientPayload:{oid}, multipart: size > 8 MB})` sends bytes directly to Blob; (3) `POST /api/orders/:id/files` registers it. All inside `S.run`, which then reloads.
- **Attachment list:** each file is a link to `/api/files/:id` (opens in a new tab) with size in KB and, for editors, a remove button with confirm.
- **Milestone cards:** coloured by state, showing week, title, pill ("Passed date", "OVERDUE" or "Due date"), the due date and, if unfinished, when the reminder starts.
- **Activity history:** the order's `log`, newest first (hidden when printing).

4.11 Users component

Loads `GET /api/users` on mount. `act(fn)` runs an action, reloads the list, and alerts any error. Each row has a role dropdown, an Active checkbox, Reset password (a `prompt`), and Delete (with confirm). The "Create user" row posts `{username, name, password, role}` and clears the fields on success.

4.12 Settings component

Edits a local copy of the settings (`c`) plus `fieldsTxt`, the default custom fields as a comma-separated string. Inputs: app name, manufacturer, default weeks, theme colour (colour picker), default extra fields, the **Open dashboard** checkbox (`public`) and the list of default milestones (via `MsEdit`). **Save settings** strips `fieldsTxt`, splits it by commas into `fields`, and calls `PUT /api/cfg`; the following reload changes the header title, colour and browser tab title. The `Users` component is rendered below.

4.13 Dashboard - every calculation

Variable / function	How it is computed and what it drives
`fyOf(date)`, `fyLbl(y)`, `CUR_FY`	The business year runs September to August: months 9-12 belong to the year of the date, months 1-8 to the previous year. 2026-09-10 -> 2026, labelled "2026-27". `CUR_FY` is the year of today.
`all`	Every order paired with its status (`ost`) and year, sorted newest PO date first. Memoised with `useMemo` on `S.orders`.
`years`	Continuous list from the earliest order year to the latest (always including the current year); feeds the From/To dropdowns and year bars.
`byYear`, `maxY`	Orders per year and the largest value, to scale the year bars (max width 50%).
`rng`, `setFrom`, `setTo`, `pick`	Selected year range (default: current year). `setFrom`/`setTo` keep From <= To. Clicking a year bar picks that single year; buttons select "All years" or "Current year".
`rows`	Orders whose year lies inside the range. Drives the cards and the stage bars.
`cnt`	Counts for the five cards: all; done; red = not done and late; green = not done and not late; soon = not done and in a reminder window (an order can be both red and soon).
`stages`, `max`	For every unfinished order, its current milestone label ("W4: Material certificates") is counted; shown as clickable bars "Open orders by current milestone".
`f` (filter), `stage`, `q` (search)	`f` is set by clicking a card or the dropdown; clicking a stage bar sets `stage`; typing sets `q`.
`vis`	The visible list: starts from `all` when searching (all years) or `rows` otherwise, then applies the status filter, the stage filter and the search text (matched against order no, customer, manufacturer ref, e-mail and all extra field values, case-insensitive).
`exp()`	CSV export of `vis`: columns for order data, status, every custom field name found in any order, and every milestone label found in any order (cell = "Passed date", "OVERDUE" or "Due date"). Values are quoted with doubled quotes, a UTF-8 BOM is prepended so Excel reads accents correctly, and a temporary link downloads `orders-YYYY-MM-DD.csv`.
`Stat`	One clickable card; highlighted when its filter is active.
Order row	Colour by `cls()`, "Next: W.. title, due date" line from `s.ms[s.cur]`, the `Track` strip; clicking opens `Detail`.

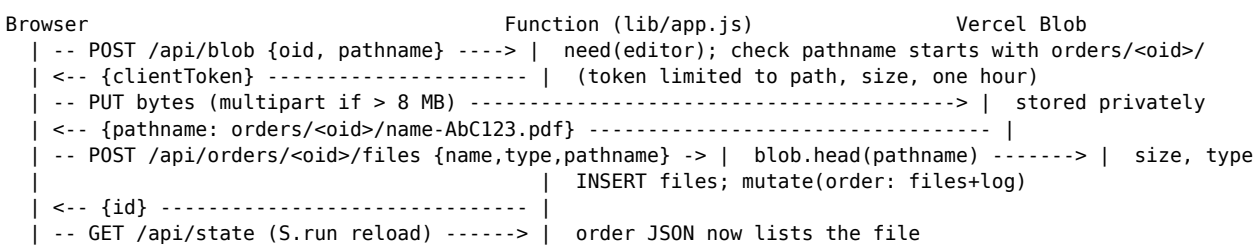
4.14 App - the shell

- **State:** `tab` (`dash`, `new`, `set`), `login`, `acct` (modal flags), `sel` (id of the order shown in Detail), `edit` (order object being edited).
- `run(fn)`**:** the universal action wrapper. It runs `fn`; on error shows an alert (a friendly message for 409: "Someone else changed this order. The latest version has been loaded; please repeat your change.", and returns to the dashboard on 401); then **always** calls `app.load()`.
- `S = {...app, run}` is passed as the prop `S` to child components so they share data and `run`.
- **Branding effect:** when `brand` (settings or the locked-page brand) changes, it sets `document.title` and the CSS variable `--ac`.
- **Header/nav:** shows Dashboard (not when locked), + New order (editors and admins), Settings (admins), your name (opens Account) and Logout (which posts `/api/logout`, returns to the dashboard and reloads) or Login.
- **Main area:** "Loading..." until the first load completes; `Login` when locked; the active tab content. Modals: `Detail` when an order is selected and not being edited; `OrderForm` in a modal when editing; `Login` and `Account` modals.

5. Feature walkthroughs (end to end)

Feature	Frontend path	Backend path	Result
First deployment	-	`init()` creates tables, secret and admin from `ADMIN_USER` / `ADMIN_PASSWORD`	You can sign in as admin.
Sign in	`Login.go` -> `api POST /api/login` -> `S.load()`	throttle check -> user lookup -> `checkPw` -> `issue()` cookie	Header shows your name; role-based buttons appear.
Sign out	Logout button -> `POST /api/logout`	cookie cleared (`Max-Age=0`)	Dashboard reloads in anonymous mode (or login page if private).
Create order	New order tab -> `OrderForm.save` -> `POST /api/orders`	`need(editor)` -> `clean` -> INSERT rev 1	Dashboard, new order opens in `Detail`.
Edit order	Detail -> Edit -> `OrderForm.save` -> `PUT` with `rev`	`clean(old)` -> `diffLog` -> `casOrder`	History shows "Order details edited".
Pass milestone / remarks	`patch()` -> `PUT` with one change	same as edit; diff gives "Passed" or "remarks updated"	Milestone card turns green; counts update.
Edit conflict	Two users edit the same order; second save sends stale `rev`	`b.rev !== old.rev` or CAS fails -> 409	Alert "Someone else changed this order"; latest data loaded.
Attach file	`upload()` -> Blob client -> `POST /api/orders/:id/files`	token check -> blob stored -> `head` -> INSERT -> `mutate`	File appears in the list; log "Attached ...".
Download file	Click link -> `GET /api/files/:id`	`canRead` -> `blob.get` -> stream	PDF/image displays; other types download.
Remove file	Detail × -> `DELETE /api/files/:id`	`mutate` removes entry; blob and row deleted	Log "Removed attachment ...".
E-mail reminder	`rem()` builds `mailto:`	none	Mail client opens with the status text.
Search / filter / export	`Dashboard` computes in memory	none (data already loaded)	List narrows; CSV downloads.
Change settings	Settings -> `PUT /api/cfg`	`kvSet("cfg")`	Branding, defaults and public/private switch change.
Manage users	`Users` -> `POST/PUT/DELETE /api/users`	guards: valid role, 8+ chars, last admin	Role or access changes; `tv` bump logs the user out.
Change own password	`Account` -> `POST /api/password`	old password check, new hash, `tv`+1, new cookie	This session continues, others end.
Delete order	Detail -> Delete (admin) -> `DELETE /api/orders/:id`	blobs deleted, rows deleted	Order disappears for everyone at next refresh.
Auto refresh	`useApp` interval every 20 s	`GET /api/state`	Changes from other users appear.

5.1 Sequence: attaching a file



5.2 Sequence: concurrent edit conflict

```
Alice loads order (rev 5)        Bob loads order (rev 5)
Alice passes W4 -> PUT rev 5  -> server: rev matches, writes, rev becomes 6
Bob   passes W6 -> PUT rev 5  -> server: stored rev is 6 != 5  -> 409
Bob's browser: alert + load() -> now shows rev 6 with Alice's change; Bob repeats his action -> PUT rev 6 -> OK (rev 7)
```

6. Things to know when changing the code

- **Milestone identity** is `week|title` in both `Detail.patch` and `diffLog`. Two milestones with the same week and title will change together, and renaming one is logged as remove + add.
- **Adding an order field:** add it in `clean()` (validation, storing), in `OrderForm` (input), and wherever it is displayed (`Detail` summary, `Dashboard` search/export). Because orders are JSON, no database change is needed.
- **Adding a column or table:** edit `init()`; use `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` for existing databases.
- **Adding an endpoint:** add an `if` branch in `api()`, call `need(...)` first, validate input, use parameterised SQL, return with `send()`.
- **Status logic** lives only in `ost()`. Changing how "late" or "soon" is defined means changing that one function.
- **Who can do what** is decided only by `need()`/`canRead()` on the server. Hiding buttons in the UI is cosmetic.
- **Never put secrets in `public/`.** Everything there is downloadable by anyone.
- **Today's date** in due-date logic is the viewer's local date, so the same order may flip to overdue at slightly different moments for users in different time zones.
- **Cold starts:** the first request after idle pays for `init()` and a new database connection; later requests are fast.

Glossary

Term	Meaning
PO	Purchase order: the customer's order for valves.
Milestone	A document deadline inside an order, defined as N weeks after the PO date.
MTC	Material test certificate.
ITP / QAP	Inspection and test plan / quality assurance plan.
CAS / optimistic locking	Compare-and-swap: update only if the version number is unchanged, otherwise report a conflict.
`tv`	Token version of a user; incrementing it invalidates that user's session cookies.
Serverless function	Code that Vercel starts on demand per request; it keeps no state between requests.
Blob	Vercel's object storage for uploaded files.
UMD build	A script version of a library that defines a global variable (`React`), loadable with a plain `<script>` tag.